

How Deodapper.com works

Deep analysis generated 2026-09-22 · app booted via docker; screenshots are real captures · model claude-sonnet-5

Overview

Deodapper is a personal project (by a developer named Avishek, named for an elephant called Deodap) that turns a block of free time — classically the gap between landing at an airport and checking in to a hotel — into a self-guided, narrated driving tour. A visitor supplies a start point, an end point, a time window and a set of interests; an AI model (Claude, via the Anthropic SDK or the ``claude`` CLI) proposes a sequence of stops, the server grounds each candidate against Wikipedia and OpenStreetMap/Nominatim so nothing is invented, a routing engine (Valhalla, with OSRM as a fallback) computes drive times, and a scheduler fits the stops into the available hours — picking a lunch stop, trimming or shortening stops that don't fit, and reporting what had to be dropped. The result is an editable itinerary that can then be "prepared" for driving: a second AI pass writes short spoken scripts for every stop plus interesting "drive-by" points of interest discovered along the route corridor via Wikipedia geosearch.

The product serves two audiences via two tiers, selected on the landing page. "Create your own route" is the paid, full planning experience (called "subscriber" internally): pay-per-route via Stripe (a card hold that's only captured once a working route is delivered), \$1.99-\$4.49 depending on how many hours (and therefore how many Claude-priced stops) the day requires. "Drive a saved route" is free: visitors search a shared library of routes that paying users chose to publish, pick one, set their own date and start time (the schedule re-fits but the stops themselves are frozen because narration is tied to them), and drive it exactly like a subscriber would. A hidden "owner mode" (press-and-hold the logo, or `?owner``) lets the site's operator use the full paid flow without paying, gated by a passphrase with brute-force lockout protection.

Once a trip is "prepared," everything needed to drive it — the route geometry, the stop list, and every narration script — is bundled into a single JSON "drive package" and saved to the phone (IndexedDB). Drive mode (``drive.html``) is a installable, offline-capable PWA: it needs no server or Claude at all, only map tiles over cell data. It follows the car by GPS, matches position against the route and against geofenced narration triggers, and speaks stop introductions and drive-by facts through the phone's own text-to-speech, alongside a simple turn-by-turn voice reading from the underlying route steps (a convenience layered on top of, not a replacement for, whatever navigation app is actually driving, e.g. Google Maps on the car's screen).

Administration and accountability are handled through a password-protected ``/admin.html`` dashboard: usage analytics (page views, devices, coarse IP-based geography, event kinds) with no personally identifying data retained, a Stripe-based sales panel, a Claude cost/usage panel used to calibrate pricing, and a moderation tool to delete any published route that slipped past the automatic content filter.

Architecture

The backend is a single Express app (``server/index.js``) with no external database server — persistence is Node's built-in ``node:sqlite`` writing to ``data/deodapper.db`` on a persistent disk, holding four logical stores: the shared route library (``server/lib/library.js``), usage analytics (``server/lib/analytics.js``), Claude call/cost accounting (``server/lib/usage.js``), owner tokens and brute-force counters (``server/lib/owner.js``), and Stripe payment credits (``server/lib/pay.js``). A lightweight tier/gate layer stamps every ``/api`` request ``subscriber`` or ``free`` (``owner.tierFor``) and a ``requireAccess`` middleware enforces payment credits (``x-credit`` header) or the owner passphrase (``x-app-key``) on the AI-costing routes (``/api/plan``, ``/api/suggest``, ``/api/prepare-drive``, ``POST /api/routes``), while cheaper routes are simply rate-limited per IP (``server/lib/ratelimit.js``).

Planning logic lives in ``server/plan.js`` (orchestration + SSE streaming), ``server/claude.js`` (SDK or CLI transport to Claude), ``server/resolve.js`` (grounding candidates against ``server/wikipedia.js`` and ``server/nominatim.js``), ``server/schedule.js`` (routing via ``server/osrm.js``/Valhalla plus the trim/lunch/shorten scheduling algorithm), and ``server/narrate.js`` (drive-prep: turn-step extraction, ``server/lib/wikiGeo.js`` for corridor points-of-interest, and a single

Claude call that writes every narration script). Pure math shared between browser and server — geo distance/bearing, route projection, and pricing tiers — lives in files imported by both sides (`web/js/routeMath.js`, `web/js/pricing.js`, `web/js/format.js`, `web/js/schedule-core.js`), avoiding logic duplication between `server/lib/geo.js` and the client.

The frontend is a build-free, vanilla ES-module static app (`web/`) served either by the same Express server or via GitHub Pages for the offline PWA shell. `web/js/main.js` boots the planner screen (`plan.html`), wiring together `state.js` (central itinerary state with a subscribe/render loop), `map.js` (Leaflet), `itinerary.js` (form + stop list rendering), `actions.js` (all itinerary mutations, each debounced through `/api/schedule`), `api.js` (fetch wrapper handling auth headers, SSE parsing, and 401-triggered passphrase prompts), `pay.js` (Stripe Payment Element + credit lifecycle), and `library.js` (free-tier saved-route search). `web/js/drive.js` boots the separate drive screen entirely from a locally stored "drive package" (`storage.js`, IndexedDB), using `geofence.js`, `speech.js`, `turnVoice.js`, and `sim.js` (a simulated-drive debugging mode) — no network calls to the planning server are required once a trip is prepared. `web/sw.js` provides the service worker for offline installability.

Data flows in one direction per phase: browser → `POST /api/plan` (SSE) → Claude proposes stops → Wikipedia/Nominatim ground them → Valhalla/OSRM route them → scheduler times/trimms them → streamed itinerary back to the browser, which the user edits via `/api/schedule` (re-route/re-time only, never re-trims). When ready, `POST /api/prepare-drive` (also SSE) turns the itinerary plus corridor drive-bys into a narrated drive package, which the browser persists locally and optionally publishes (`POST /api/routes`, filtered by `server/lib/moderation.js`) to the shared library that free-tier users query via `GET /api/routes`.

Landing page presenting Deodapper and letting a visitor choose between driving a free, already-published route or paying to have Deodap plan and narrate a new one.



Deodapper

SELF-GUIDED TOUR CREATOR

Turn the hours between landing and check-in into a real tour.
Tell Deodap what you love, get a timed route of stops, and
hear the story of each place as you drive past it.

Drive a saved route

Free · pick a route others have built
and let Deodap narrate it

Create your own route

From \$1.99 a route · Deodap plans a
day around your interests and writes
the narration. You're only charged
when it's ready.

- Plan.** Start, end, the time you have, what you're into.
Deodap picks the stops and works out what fits.
- Prepare.** Every stop and the interesting places along the
way get a short spoken story.
- Drive.** Phone on the mount, Google Maps on the car screen.
Deodap talks when you get there.

Already have a trip on this phone? [Open drive mode.](#)

Payments are handled by Stripe; Deodapper keeps a payment reference, never your card or contact
details. Usage stats are IP-level only. [Terms & refunds](#) · [Privacy](#) · [Contact](#)

Controls

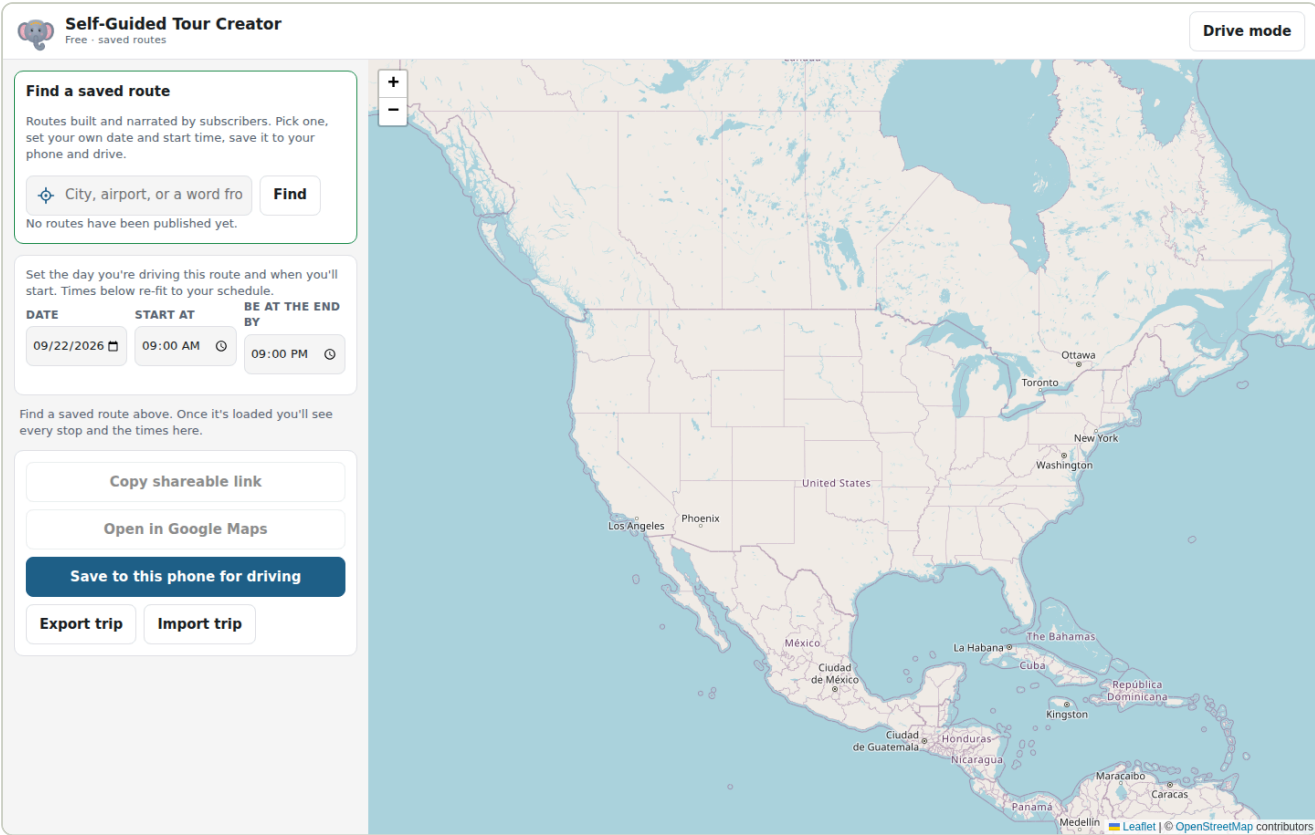
CONTROL	WHAT IT DOES
"Drive a saved route" (a.choice.free)	links to `plan.html?tier=free`; `web/js/config.js`'s `tier()` reads `?tier=free` from the URL, persists it to `localStorage`, and `main.js` adds `body.free`, which the plan page's CSS uses to hide subscriber-only controls and show the "Find a saved route" library search instead.
"Create your own route" (a.choice.sub)	links to `plan.html?tier=create`; `config.tier()` maps the public label `"create"` to the internal tier `"subscriber"`, enabling the full AI planner form, Suggest more, prepare-drive, and the Stripe pay-per-route flow (`web/js/pay.js`).
"Open drive mode" (a, footer)	links directly to `drive.html`; on load, `web/js/drive.js`'s `loadTrip()` looks up `storage.getActiveTripId()`/`storage.listTrips()` in IndexedDB to resume whatever trip is already saved on the phone.
"Terms & refunds" (a)	links to `terms.html`, the pricing/refund/legal page.

"Privacy"
(a) links to `privacy.html`, the data-retention/legal page.

"Contact"
(a) `mailto:support@deodapper.com`, the address required for Stripe's activation review and used throughout the legal pages.

Page: /plan.html

The combined route planner/editor for paying "subscriber" users and the saved-route finder for "free" users; also where a prepared trip is exported or handed off to drive mode.



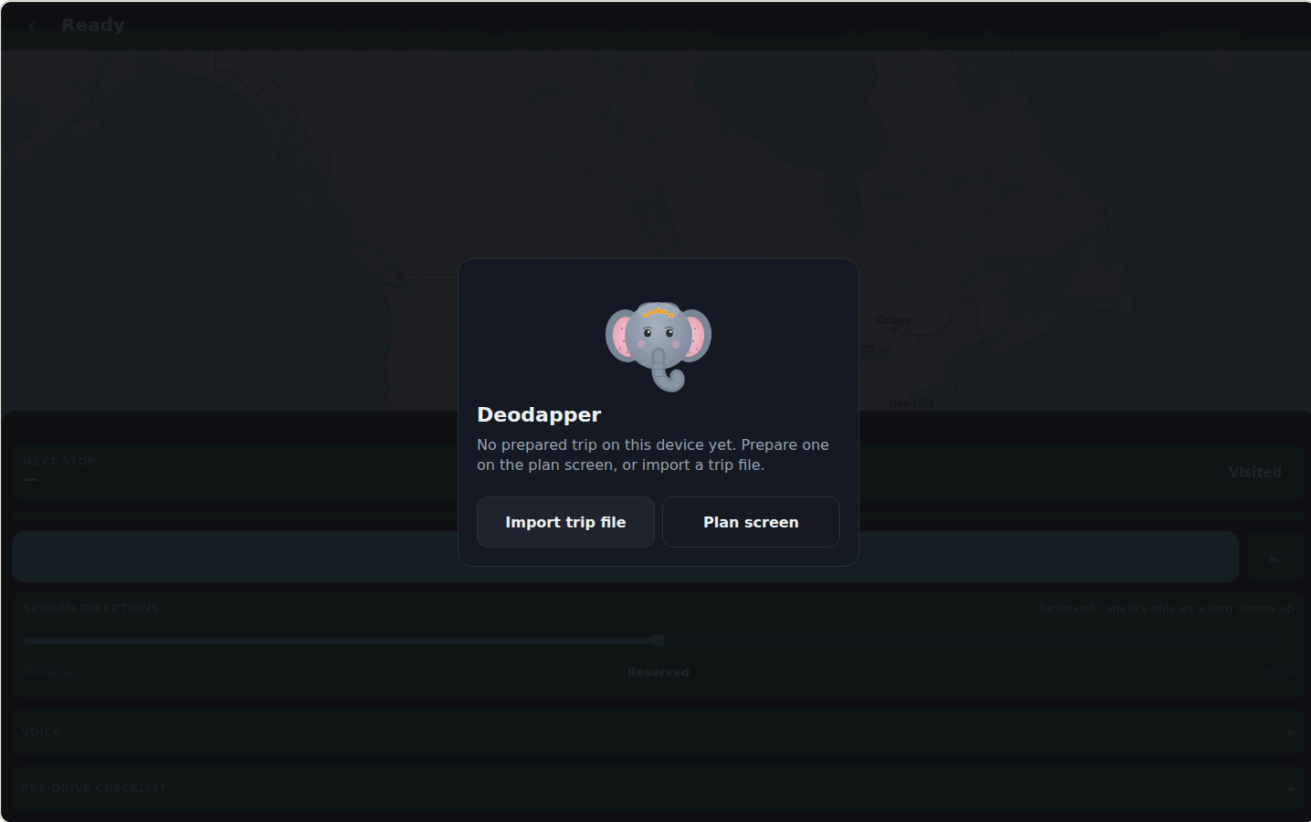
Controls

CONTROL	WHAT IT DOES
"Self-Guided Tour Creator" brand link (a)	returns to `index.html`; the adjacent `#tier-badge` span is set by `main.js` to "Free · saved routes" or "Create your own route" depending on `isFree()`.
"Drive mode" (a#drive-link)	navigates to `drive.html` to resume the active or a picked trip.
"Zoom in" / "Zoom out" (a)	Leaflet's built-in zoom control buttons on the map created in `map.init()`.
"Leaflet" / "OpenStreet Map" (a)	standard tile/library attribution links rendered by Leaflet, not app logic.
"Routes near my location"	in `library.js`'s bind handler, requests browser geolocation and calls the library search (`GET /api/routes?near=lat,lon`) via `server/lib/library.js`'s `search()`, which keeps routes whose start or end lies within the radius.

(button#library-near)	
City/airport/word search input (input#library-query)	free-text query bound to the "Find" button; matched server-side against title, description, region, interests and stop names in <code>`library.search()`</code> .
"Find" (button#library-search)	submits the query text to <code>`GET /api/routes?q=...`</code> , rendering results into <code>`#library-results`</code> .
Date field (input[date])	sets <code>`it.date`</code> in <code>`state.js`</code> ; for a free-tier user this is "set the day you're driving this route," re-timing the fixed stop list.
Start/arrival and deadline time fields (input[time])	set <code>`arrivalTime`</code> / <code>`deadline`</code> ; changes are debounced through <code>`actions.reschedule()`</code> , which calls <code>`POST /api/schedule`</code> to re-route and re-time (never re-trims) and, for subscribers, refreshes the Stripe price quote via <code>`pay.renderPrice()`</code> .
"Copy shareable link" (button)	<code>`share.js`</code> 's bind handler encodes the current itinerary into a <code>`#i=...`</code> URL hash (the inverse of <code>`share.loadFromHash()`</code> used at boot to restore a trip from a shared link).
"Open in Google Maps" (button)	builds and opens a Google Maps URL from the current start/end/stops so the visitor can hand real turn-by-turn navigation to Google Maps while Deodapper narrates.
"Save to this phone for driving" (button)	<code>`drivePrep.js`</code> / <code>`storage.js`</code> logic that merges the prepared narration package into IndexedDB (<code>`storage.saveTrip`</code>), applying the same <code>`planMatch.js`</code> check used for publishing so stale or edited-but-unprepared plans aren't saved under the wrong stops.
"Export trip" (button)	downloads the current drive package as a JSON file (the same shape as <code>`samples/houston-drive-package.json`</code>), for manual transfer or backup.

Page: /drive.html

The offline-capable driving screen: follows the car by GPS and narrates stops and points of interest from a trip package already saved on the phone, with no server or Claude dependency during the drive.



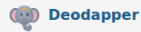
Controls

CONTROL	WHAT IT DOES
"Leaflet" / "OpenStreet Map" (a)	map/tile attribution, standard Leaflet output.
"Back to plan" (a.back)	href is set dynamically in `usePackage()` to `plan.html?trip=<tripId>`, reopening the same trip for editing.
Sheet grip / "Show or hide the panel" (div#sheet-grip)	drag/tap handle that expands or collapses the bottom sheet containing the stop list and settings.
"Visited" (button#skip-stop)	manually marks the current next stop as visited, advancing `state.nextStopIdx` and updating the geofence's visited set (persisted via `storage.getDriveState`/save).

"Start drive" (button#start -btn)	begins <code>`navigator.geolocation.watchPosition`</code> tracking, engages the wake lock, and starts the narration/geofence engine (replaced by a hidden "End drive" button once running).
"🔊" mute toggle (button#mute- btn)	toggles narration audio via <code>`speech.js`</code> , reflected in its <code>`aria-pressed`</code> state.
Spoken- directions slider (input[range] #turn-voice)	three-position control (Talkative / Reserved / Mute) feeding <code>`turnVoice.js`</code> 's <code>`createTurnVoice`</code> / <code>`saveTurnMode`</code> , governing how verbose the spoken turn-by-turn cues are.
"Voice ►" (summary#voice- section)	expands/collapses the narration voice settings panel.
Narration voice (select#voice- preset)	chooses among <code>`VOICE_PRESETS`</code> defined in <code>`speech.js`</code> , matched to the device's available TTS voices via <code>`matchVoice`</code> .
"Test" (button#voice- test)	plays a sample narration line in the currently selected voice.
"Pre-drive checklist ►" (summary)	expands a static checklist (location permissions, ringer, phone mount, etc.) with no server interaction.
"Plan screen" (a, overlay)	shown when no trip is loaded; links to <code>`plan.html`</code> to start or resume planning, alongside an "Import trip file" control (input#overlay-import) that reads a JSON package directly into IndexedDB via <code>`storage.js`</code> .

Page: /terms.html

The legal/pricing page covering what Deodapper is, route pricing, the hold-then-capture payment model, and the refund policy.



Terms and refunds

Effective 21 September 2026

What Deodapper is

Deodapper plans a self-guided driving day between two places, around the things you're interested in, and narrates it out loud as you drive. The stops and the stories are written on demand by an AI model (Claude, by Anthropic) from public sources such as Wikipedia and OpenStreetMap. Deodapper is run by an individual in the United States, not a company with a support desk, so please read this page before you pay.

Free and paid

Drive a saved route is free: pick a route someone else created, set your own day and time, save it to your phone and drive it. **Create your own route** is paid per route. There is no subscription and no account.

ROUTE	TIME WINDOW	PRICE
Short outing	up to 3 hours	\$1.99
Half day	3 to 6 hours	\$2.99
Full day	over 6 hours	\$4.49

Prices are in US dollars. Sales tax, where it applies, is added at checkout. One payment covers, for the start and end you chose: up to three plans (the first plus two re-plans, as long as the day stays within the tier you paid for), edits to the stops, spoken narration for the whole drive, and the option to publish the route for free-tier drivers. It stays usable for 30 days.

How payment works

Payments are processed by [Stripe](#). When you press Plan, Stripe places a **hold** on your card for the price shown. Nothing is charged yet. Your route is planned, and only when it is ready is the hold **captured**. If planning fails, or you cancel before it finishes, the hold is released and you are not charged. Uncaptured holds also expire on their own within a week. Your card details go to Stripe directly and never reach Deodapper; we keep only a payment reference.

Refunds

- **You were not charged** for anything that didn't produce a route. That is the normal case for a failed or cancelled plan, and it happens automatically.
- **Delivered routes are not refundable** as a rule, because each one is generated for you on demand and costs us to make. The stops are based on public information and may not match your taste; that is not a defect.
- **We will refund** a duplicate or accidental purchase, and a route that was genuinely unusable (for example, the planner returned no stops or the narration was missing) if you tell us within 7 days. Contact us with the date of the payment and the start and end of the route. Refunds go back to the original card and can take 5 to 10 days to appear.

Use it sensibly

You are the driver. Deodapper is not a navigation system: the turn banner and spoken directions are a convenience, and your car's or phone's navigation should lead. Obey traffic laws, keep your eyes on the road, and set up the phone before you move off. Stop information, opening hours and stories come from an AI model working from public sources and can be wrong, out of date or incomplete; check anything that matters before you rely on it. Deodapper is provided as is, and our liability to you is limited to the amount you paid for the route in question.

Publishing a route

If you publish a route, you allow other Deodapper users to load and drive it, and you confirm that the title and description you wrote are yours to share. We filter published text for markup, code and abusive language, and we may remove a route at any time. Don't publish addresses that identify a private home; the app refuses obvious ones.

Availability

Planning depends on outside services (Wikipedia, OpenStreetMap routing, the AI model). They are sometimes slow or unavailable, and so, at those moments, is Deodapper. We don't promise any particular uptime. Drive mode works offline once a route is saved to your phone.

Changes and contact

We may update these terms; the date at the top tells you when. Questions, refund requests and takedown requests: support@deodapper.com. See also our [privacy policy](#).

[Back to Deodapper](#)

Controls

CONTROL	WHAT IT DOES
"Deodapper" (a.doc-home, logo+text)	returns to `index.html`.
"Stripe" (a)	external link to stripe.com, referenced where payment processing is described.
"support@deodapper.com" (a, mailto)	contact address for refund/takedown requests.
"privacy policy" (a)	links to `privacy.html`.
"Back to Deodapper" (a)	returns to `index.html`.

Page: /privacy.html

The privacy policy detailing exactly what data Deodapper stores (usage analytics, payment references, published-route content) versus what it explicitly never stores (card data, names, emails, GPS location, cookies).

Privacy

Effective 21 September 2026

The short version: Deodapper has no accounts, never sees your card, and never stores your name or email. It keeps rough usage statistics for 90 days and a reference number for each payment. Your location while driving stays on your phone.

What we keep

- **Usage statistics.** For each page view and action: the time, your IP address, the approximate city derived from it, your device type and browser family, and what you did (planned, prepared, published, searched). Nothing you type into forms. Kept for 90 days, then deleted.
- **Payment references.** When you pay for a route: a Stripe payment id, the price tier, the amount, its status (held, charged, released), timestamps, the start and end area of the route (rounded to about a kilometre), the IP address that paid, and a random id your browser generated. This lets us honour your purchase. It is kept for 30 days after the credit expires, and longer only in Stripe's own records.
- **Your route while it's being made.** The start, end, times and interests you enter are processed on our server to plan and narrate the route, then returned to your phone. We don't store your route on the server unless you publish it.
- **Published routes.** If you choose to publish, the route (title, description, stops, narration, start and end places) is stored and shown to other users. Don't include personal details; we screen for street addresses.
- **Your Claude and payment usage in aggregate,** for our own costs and accounting, without anything that identifies you.

What we don't keep

- Card numbers, expiry dates or security codes. They go from your browser straight to Stripe.
- Names, email addresses or phone numbers. If you enter them on the payment form, Stripe holds them; we don't receive them.
- Your GPS position while driving. Drive mode runs on your phone and works offline; your location is never sent to us.
- Cookies. We use your browser's local storage for settings, your saved routes, your route credit and the random device id, and nothing follows you across sites.

Services we rely on

Building a route involves other companies, each seeing only what it needs:

- **Stripe** processes payments and holds whatever you enter on the payment form. See [Stripe's privacy policy](#).
- **Anthropic (Claude)** receives your start, end, times and interests, and the names and Wikipedia summaries of candidate stops, to propose stops and write narration. It receives no payment or contact details.
- **Wikipedia / Wikimedia** and **OpenStreetMap's Nominatim** are queried by our server for place information. **Photon (komoot)** and OpenStreetMap map tiles are fetched by your browser directly for the search box and the map, so those services see your IP address.
- **Valhalla (FOSSGIS) and OSRM** compute the driving route from the coordinates involved.
- **ipwho.is** turns IP addresses into an approximate city for our statistics.
- **Render** hosts the server and **Cloudflare** serves the domain. Data is stored in the United States.

Your choices

Email support@deodapper.com to ask what we hold about an IP address or a payment, or to have it deleted; give us the approximate date and, for payments, the last four digits of the card so we can find it in Stripe. Unpublishing a route you published is also a matter of asking. Deodapper is not directed at children under 13.

Changes

If this page changes, the date at the top changes with it. Our [terms and refunds](#) page covers the paid features.

[Back to Deodapper](#)

Controls

CONTROL

WHAT IT DOES

"Deodapper" (a.doc-home, logo+text)	returns to `index.html`.
"Stripe's privacy policy" (a)	external link to stripe.com/privacy.
"support@deodapper.com" (a, mailto)	contact address for data requests/deletion.
"terms and refunds" (a)	links to `terms.html`.
"Back to Deodapper" (a)	returns to `index.html`.

Page: /index.html

Identical document to the site root `` (the same static `web/index.html` file); serves as the explicit filename form of the landing page.



Deodapper

SELF-GUIDED TOUR CREATOR

Turn the hours between landing and check-in into a real tour.
Tell Deodap what you love, get a timed route of stops, and
hear the story of each place as you drive past it.

Drive a saved route

Free · pick a route others have built
and let Deodap narrate it

Create your own route

From \$1.99 a route · Deodap plans a
day around your interests and writes
the narration. You're only charged
when it's ready.

- Plan.** Start, end, the time you have, what you're into.
Deodap picks the stops and works out what fits.
- Prepare.** Every stop and the interesting places along the
way get a short spoken story.
- Drive.** Phone on the mount, Google Maps on the car screen.
Deodap talks when you get there.

Already have a trip on this phone? [Open drive mode.](#)

Payments are handled by Stripe; Deodapper keeps a payment reference, never your card or contact
details. Usage stats are IP-level only. [Terms & refunds](#) · [Privacy](#) · [Contact](#)

Controls

CONTROL	WHAT IT DOES
"Drive a saved route" (a.choice.free)	same as on ``; links to `plan.html?tier=free`.
"Create your own route" (a.choice.sub)	same as on ``; links to `plan.html?tier=create` (mapped to the `subscriber` tier by `config.tier()`).
"Open drive mode" (a)	same as on ``; links to `drive.html`, resuming any trip already saved via `storage.js`.
"Terms & refunds" (a)	links to `terms.html`.
"Privacy" (a)	links to `privacy.html`.
"Contact" (a)	`mailto:support@deodapper.com`.